



**T.C.
BURSA ULUDAĞ ÜNİVERSİTESİ
KARACABEY MESLEK YÜKSEKOKULU**

PROJE ADI

CUBE JUMP

Projeyi Hazırlayan

Melike Duman-262484053

Proje Danışmanı

Öğr. Gör. Koray ÇOŞKUN

Bilgisayar Teknolojileri Bölümü

Bilgisayar Programcılığı Programı

Haziran, 2026

Bursa

ÖZET

Unity oyun motoru ve C# programlama dili kullanılarak, oyuncu reflekslerini ön plana çıkaran "Cube Jump" adlı 2D runner (koşu) türünde eğlenceli bir oyun geliştirmektir. Projenin yöntem kısmında, nesne yönelimli programlama prensipleri benimsenmiş; karakterin hareket ve zıplama mekanikleri hassas fizik kuralları ve çarpışma algılama kodlarıyla hazırlanmıştır. Sürekli akış sağlayan sahne yönetimi, dinamik engel üretimi ve kullanıcı dostu arayüz tasarımları modüler bir yapıda kurgulanmıştır. Geliştirme sürecinde sekiz haftalık proje takvimi ve sistem tasarımı ilkeleri titizlikle uygulanarak yazılımın sorunsuz çalışması hedeflenmiştir. Sürecin sonucunda, her cihazda takılmadan akıcı bir şekilde çalışan, oynanabilirliği yüksek ve eğlenceli bir oyun başarıyla tamamlanmıştır. Ayrıca projenin temiz ve esnek kod yapısı sayesinde, gelecekte oyuna yeni seviyeler, engeller ve özellikler eklemek son derece kolay bir hale getirilmiştir.

İçindekiler Tablosu

1.	GİRİŞ.....	1
1.1.	Projenin Konusu	1
1.2.	Projenin Amacı	1
1.3.	Projenin Kapsamı	1
1.4.	Projenin Tanımı	1
2.	DETAYLI GELİŞME.....	2
2.1.	Planlama	2
2.1.1.	Proje Planı	2
2.1.1.1	Proje Kapsamı	2
2.1.2.	Proje Kaynaklarının Belirlenmesi	2
2.1.3.	Proje Maliyetlerinin Kestirilmesi	3
2.1.4.	Proje Ekip Yapısının Oluşturulması	3
2.1.5.	Ayrıntılı Proje Planı.....	3
2.1.6.	Projenin İzlenmesi.....	5
2.1.6.1.	Proje Zaman-İş Planı	5
2.1.7.	Tercih Edilen Süreç Modeli	5
2.1.8.	Gant Şeması İle Haftalık İlerleyiş.....	6
2.2.	Çözümleme/Analiz	6
2.2.1.	Analiz Raporu	6
2.2.2.	Düz Metin Açıklamaları	6
2.2.3.	Karar Tablosu	7
2.2.4.	Veri Akış Şeması	8
2.3.	Tasarım.....	8
2.3.1.	Kağıt Prototip	8
2.3.2.	Nesene İlişki Modeli,Veritabanı Tabloları,Veri Sözlüğü.....	10
2.4.	Gerçekleştirme/Kodlama	13
2.4.1.	Kullanılan Programlama Dilleri.....	13
2.4.2.	Uygulama Mimarisi	13
2.4.3.	Kodlama Süreci.....	14
2.5.	Bakım/Destek.....	15
2.5.1.	Olası Güncelleme Süreçleri	15
2.5.2.	Hata Düzeltme Mekanizmaları.....	15
2.5.3.	Sistemin Sürdürülebilirliği	16

3.	SONUÇ	16
3.1.	Elde Edilen Bulgular.....	16
3.2.	Proje Hedefine Ulařtımı	17
3.3.	Karřılasılan Sorunlar ve Çözüm Önerileri	17
4.	KAYNAKÇA	18
5.	UYGULAMAYA AİT EKRAN GÖRÜNTÜLERİ	19
6.	UYGULAMAYA AİT KODLAR	21

1. GİRİŞ

1.1. Projenin Konusu

Bu projenin konusu, dijital oyun sektöründe popüler bir alt tür olan 2D runner (sonsuz koşu) mekaniklerinin modern yazılım geliştirme prensipleriyle ele alınması ve "Cube Jump" adlı oynanabilir bir oyun prototipinin üretilmesidir. Çalışma kapsamında, minimal bir küp karakterinin dinamik olarak zorlaşan bir platform üzerinde engelleri aşma mücadelesi temel izlek olarak belirlenmiştir. Proje; nesne hareketleri, kullanıcı girdilerinin anlık işlenmesi, çarpışma testleri ve sahne yönetimi gibi temel oyun programlama unsurlarının Unity motoru ve C# dili kullanılarak optimize bir şekilde bir araya getirilmesini konu edinmektedir.

1.2. Projenin Amacı

Bu projenin temel amacı; Unity oyun motoru ve C# programlama dili imkanlarından yararlanarak, kullanıcı reflekslerine dayalı, akıcı ve optimizasyonu yüksek bir 2D runner (koşu) oyunu geliştirmektir. Teknik açıdan amaç; nesne yönelimli programlama (OOP) prensiplerine uygun, temiz, esnek ve modüler bir kod mimarisi kurarak oyun içi mekaniklerin (fizik, çarpışma algılama ve sahne yönetimi) sorunsuz çalışmasını sağlamaktır. Tasarım ve oynanış açısından ise; dengeli bir zorluk eğrisine sahip, kullanıcı dostu arayüzüyle oyuncuyu içine çeken ve farklı donanımlara sahip cihazlarda kararlı bir performans (FPS) sunan bir mezuniyet projesi ortaya koymak hedeflenmiştir.

1.3. Projenin Kapsamı

Bu projenin kapsamı, "Cube Jump" adlı 2D runner oyununun sistem analizi, tasarımı ve kodlama süreçleriyle sınırlandırılmıştır. Proje içerisinde; Unity ortamında C# dili kullanılarak karakterin hareket ve zıplama mekanikleri, dinamik engel üretimi, skor sistemi, kullanıcı arayüzü (UI) ve sahne yönetimi sıfırdan geliştirilmiştir. Sekiz haftalık takvim planına uygun olarak, 2D minimalist görsel varlıkların entegrasyonu ve oyun içi zorluk dengesinin kurulması sağlanmıştır.

1.4. Projenin Tanımı

"Cube Jump", Unity oyun motoru ve C# programlama dili kullanılarak geliştirilen, minimalist tasarıma sahip iki boyutlu bir sonsuz koşu oyunudur. Proje, temel olarak bir küp karakterinin dinamik olarak ilerleyen bir platform üzerinde karşısına çıkan engelleri zıplayarak aşması ve en yüksek skora ulaşması üzerine kurulmuş bir yazılım uygulamasıdır.

2. DETAYLI GELİŞME

2.1. Planlama

2.1.1.Proje Planı

2.1.1.1. Proje Kapsamı

Bu bölümde projenin geliştirme planını genel hatlarıyla açıklamaktayım. Öncelikle projenin türüne ve hangi platform üzerinde çalışacağına karar vermem gerekiyordu. Geniş bir oyuncu kitlesine hitap edebilecek, dinamik ve akıcı bir oynanış sunan "Cube Jump" adında 2D runner (koşu) türünde bir oyun geliştirmeyi hedefledim. Bu doğrultuda projenin teknik altyapısını ve kullanacağım teknolojileri belirledim. 2D oyun geliştirme sürecinde sunduğu güçlü araçlar, esnek fizik motoru ve hazır bileşen desteği nedeniyle oyun motoru olarak Unity'yi tercih ettim. Oyun içi mekanikleri, karakter hareketlerini ve sistem yönetimini kodlamak için ise C# programlama dilini kullandım. Kodlama ve tasarım sürecine başlamadan önce projenin görsel ve mantıksal yapısını netleştirmek amacıyla hazırlıklar yaptım. Oyunun görsel dünyasını oluşturacak asset'lerin (görsel varlıkların) seçimi, sprite editör yardımıyla parçalanması ve sahne tasarımları için Pinterest platformlardan yararlanarak basit prototipler ve yerleşim planları hazırladım. Geliştirme sürecini daha düzenli ilerletebilmek için projeyi iki temel aşamaya ayırdım: oyun mekaniklerinin kodlanması (scripting) ve görsel/sahne yönetimi. Öncelikle oyunun temelini oluşturan karakter hareketleri, engel mekanikleri, skor sistemi ve sahne döngülerini C# scriptleri ile backend mantığında kurguladım; ardından bu kodları Unity arayüzündeki görsel öğeler ve animasyonlarla entegre ettim. Hazırda gerçekleştireceğim proje teslimi öncesinde, uygulamanın hatasız ve akıcı çalışmasını sağlamak amacıyla çeşitli testler gerçekleştirmeyi planladım. Bu kapsamda oyunu hem Unity editör ortamında farklı senaryolarla test etmeyi hem de potansiyel oyunculara deneyterek kullanıcı deneyimini, oyun zorluk dengesini ve performansını gözlemlemeyi hedefledim.

2.1.2. Proje Kaynaklarının Belirlenmesi

Bu proje, tek başıma geliştirdiğim bir 2D oyun projesidir. Proje planına göre haziran ayındaki teslim tarihine kadar olan 8 haftalık süreç içerisinde tamamlanması beklenmektedir. Tasarım, görsel düzenleme ve kodlama süreçleri bir adet dizüstü bilgisayar kullanılarak yürütülmektedir. Test sürecinde ise oyunun farklı donanım özelliklerine, bilgisayar ekran çözünürlüklerine, klavye kontrollerine ve performans dengesine bakılabilmesi amacıyla 2 farklı bilgisayar (PC) üzerinden testler gerçekleştirilecektir. Projenin geliştirme, kodlama ve ilk test süreçleri Unity motoru üzerinde yerel olarak gerçekleştirileceği için bu aşamada herhangi bir yazılım veya motor ücreti ödenmeyecektir. Projenin profesyonel olarak dijital ortamda internet üzerinden bilgisayar kullanıcılarına ulaştırılabilmesi ve yayınlanabilmesi için PC oyun platformları (Itch.io, Steam vb.) kullanılacaktır. Bu aşamada bağımsız geliştiriciler için ücretsiz yayınlama imkanı sunan Itch.io gibi platformlar tercih edilirse herhangi bir maliyet oluşmayacaktır; ancak

oyunun dünyanın en büyük PC oyun mağazası olan Steam üzerinden profesyonel olarak yayınlanması istenirse, platformun güncel kuralları gereği oyun başına tek seferlik alınan yaklaşık 100\$'lık bir mağaza giriş ücreti (Steam Direct ücreti) maliyet olarak karşımıza çıkacaktır.

2.1.3. Proje Maliyetlerinin Kestirilmesi

"Cube Jump" projesi akademik bir bitirme çalışması kapsamında tek bir geliştirici tarafından yerel imkanlarla yürütüldüğü için geliştirme sürecindeki toplam fiili maliyeti 0 TL olarak gerçekleştirmiştir. Geliştirme, tasarım ve test aşamalarında halihazırda var olan kişisel bilgisayar kullanılırken; oyun motoru olarak Unity'nin ücretsiz "Personal" sürümünden, görsel öğeler için ise Pinterest ve internet üzerindeki telifsiz asset havuzlarından yararlanılarak yazılım ve tasarım maliyeti oluşması engellenmiştir. Projenin tamamlanmasının ardından PC kullanıcılarına ulaştırılması sürecinde, Itch.io gibi bağımsız platformlar üzerinden ücretsiz yayınlama seçeneği öngörülmüş; oyunun gelecekte Steam gibi profesyonel bir mağazada listelenmesi senaryosunda ise tek seferlik yaklaşık 100\$'lık bir mağaza giriş ücretinin potansiyel maliyet olarak karşımıza çıkacağı kestirilmiştir.

2.1.4. Proje Ekip Yapısının Oluşturulması

Proje üzerinde tek başıma çalıştım. Proje planlamasını bunu göz önünde bulundurarak yaptım.

2.1.5. Ayrıntılı Proje Planı

Analiz:

"Cube Jump" projesinin geliştirme sürecine başlanmadan önce hedef kitle, oyun mekanikleri ve teknik altyapıyı şekillendirmek adına detaylı bir analiz süreci yürütülmüştür. Refleks tabanlı ve akıcı bir 2D runner oyunu hedeflendiğinden, oyunun düşük donanımlı bilgisayarlarda bile yüksek performansla çalışması temel gereksinim olarak belirlenmiştir. Oyun mekanikleri kapsamında, ana karakterin PC platformuna uygun klavye kontrolleriyle yönetilmesi, engellerin dinamik akışı, skor sistemi ve çarpışma algoritmaları mantıksal olarak kurgulanmıştır. Kullanıcı deneyimini artırmak adına ana menü, oyun içi skor ekranı ve oyun sonu paneli olmak üzere üç aşamalı minimalist bir arayüz yapısı analiz edilmiştir. Teknik tarafta ise Unity'nin 2D fizik bileşenleri (Rigidbody2D, Collider) ve C# dilinin nesne yönelimli yapısı incelenerek, oyun içi sahnelerin ve kod scriptlerinin belleği yormayacak şekilde optimize yönetilmesi planlanmıştır.

Tasarım:

"Cube Jump" projesinin tasarım sürecinde, analiz aşamasında belirlenen kriterler doğrultusunda görsel, mantıksal ve mimari taslaklar oluşturulmuştur. Görsel tasarım kapsamında, Canva ve Pinterest'teki minimalist öğelerden yararlanılarak ana menü, oyun içi skor göstergesi ve oyun sonu paneli için kullanıcı dostu arayüz yerleşimleri ve renk paletleri belirlenmiştir. Oyun tasarımı boyutunda, sonsuz koşu (endless runner) mekaniğine uygun olarak engellerin yerleşimi, ekrana geliş sıklıkları ve zamanla hızı artıran dinamik bir zorluk eğrisi kurgulanmıştır. Yazılımsal tasarım tarafında ise Unity mimarisine uygun olarak karakter kontrolleri, engel üretici (spawner) ve genel oyun yöneticisi (Game Manager) gibi temel C# scriptlerinin birbiriyle optimize çalışmasını sağlayan nesne yönelimli mimari modellenmiştir.

Kodlama:

"Cube Jump" projesinin kodlama aşaması, Unity oyun motoru bünyesinde nesne yönelimli programlama ilkelerine uygun olarak C# dili ile gerçekleştirilmiştir. Oyunun temel oynanış mekaniklerini yönetmek adına kod yapısı işlevlerine göre bağımsız scriptlere bölünmüştür. Bu kapsamda; ana karakterin PC klavyesindeki boşluk tuşuna basıldığında zıplama hareketini gerçekleştirmesini sağlayan fizik tabanlı (Rigidbody2D) karakter kontrol kodları yazılmıştır. Oyunun sonsuz döngüde ilerleyebilmesi için engelleri belirli zaman aralıklarıyla ve rastgele konumlarda ekrana üreten bir engel üretici algoritması kurgulanmıştır. Karakterin engellere çarpıp elenmesini algılamak amacıyla Unity'nin çarpışma tetikleyicileri (OnTriggerEnter2D/OnCollisionEnter2D) kullanılmış ve oyun bittiğinde "Game Over" panelini ekrana getiren, skor tablosunu güncelleyen ve yeniden başlatmayı sağlayan genel bir oyun yöneticisi (Game Manager) ile sahne yönetim (SceneManagement) mimarisi kodlanarak projenin temel yazılım altyapısı tamamlanmıştır.

Test:

Projenin kodlama sürecini tamamladıktan sonra test ve optimizasyon aşamasına geçmeyi planlamaktayım. Oyunun farklı donanım özelliklerine sahip bilgisayarlardaki performansını, akıcılığını (FPS) ve ekran çözünürlüğü uyumluluğunu görebilmek amacıyla projeyi yaklaşık 3 farklı bilgisayar (PC) üzerinden test etmeyi hedeflemekteyim. Ayrıca yakın arkadaşlarımdan oyunu deneyimlemelerini isteyerek klavye kontrollerinin tepki süresini ve oyunun zorluk dengesini kullanıcı gözünden test edeceğim. Projeyi tek başıma geliştirdiğim için test sürecinde ekstra dikkatli davranarak fizik tabanlı takılmaları ve kod hatalarını (bugları) titizlikle tespit edip düzeltecek, oyunu kararlı bir hale getireceğim. Son aşamada ise gelecekteki olası güncellemeler ve dosya bozulmalarına karşı kaynak kodların düzenli olarak yedeklenmesini sağlayacağım.

2.1.6. Projenin İzlenmesi

2.1.6.1. Proje Zaman-İş Planı

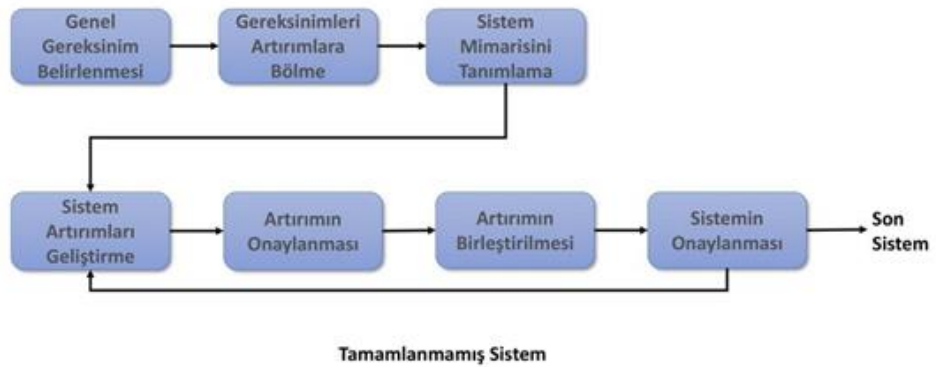
Hafta	Planlanan Faaliyetler ve Kilometre Taşları
1. Hafta	Analiz ve Gereksinim Belirlenmesi: Proje konusunun netleştirilmesi, teknik gereksinimlerin analizi ve kullanılacak varlıkların (assets) belirlenmesi.
2. Hafta	Tasarım ve Prototipleme: Oyun sahnelerinin (Main Menu, Gameplay) taslak çizimleri, minimalist renk paletinin seçimi ve UI (arayüz) tasarımı.
3. Hafta	Temel Mekaniklerin Kodlanması: Unity ortamında fizik motoru ayarları, karakterin zıplama ve hareket algoritmalarının C# ile yazılması.
4. Hafta	Görsel Mimari ve Paralaks Entegrasyonu: Arka plan katmanlarının (6 katmanlı yapı) sisteme dahil edilmesi ve derinlik etkisinin (parallax) kodlanması.
5. Hafta	Arayüz (UI) ve Sahne Yönetimi: Giriş ekranı, skor tablosu ve sahne geçiş sistemlerinin (Scene Management) entegrasyonu.
6. Hafta	Hata Ayıklama ve Optimizasyon: Karşılaşılan çalışma zamanı hatalarının (NullReference vb.) giderilmesi ve farklı ekran çözünürlükleri için uyumluluk testleri.
7. Hafta	İşitsel Entegrasyon ve Son Rötuşlar: Ses efektlerinin (SFX) ve arka plan müziklerinin eklenmesi, oynanış dengesinin (game balance) kurulması.
8. Hafta	Final Testleri ve Raporlama: Projenin son testlerinin yapılması, çıktıların alınması ve proje sonuç raporunun tamamlanması.

Tablo 1 - Zaman-İş Plan Tablosu

2.1.7. Tercih Edilen Süreç Modeli

Bu projede yazılım geliştirme sürecini planlarken Arttırımlı geliştirme modelini kullanmayı tercih ettim.

Arttırımsal Geliştirme Modeli



Resim 1-Arttırımsal Süreç Modeli [1]

2.1.8. Gant Şeması İle Haftalık İlerleyiş

Haftalık Çizelge	H1	H2	H3	H4	H5	H6	H7	H8
1. Analiz ve Gereksinim Belirlenmesi	■							
2. Tasarım ve Görsel Asset Seçimi		■						
3. Temel Mekanikler ve Karakter Kodları			■					
4. Arka Plan ve Paralaks Katman Yönetimi				■				
5. Arayüz (UI) ve Menü Tasarımları					■			
6. Sistem Entegrasyonu ve Hata Ayıklama						■		
7. Ses Entegrasyonu ve Kontrolü							■	
8. Final Testleri ve Rapor Teslimi								■

Şekil 1-Gant Şeması İle Haftalık İlerleyiş

2.2.Çözümleme/Analiz

2.2.1. Analiz Raporu

“Cube Jump” projesinin bilgisayar platformu (PC) için, minimum sistem gereksinimlerine sahip donanımlarda dahi akıcı ve yüksek performansla çalışabilecek bir 2D sonsuz koşu (endless runner) oyunu olmasına karar verilmiştir. Kullanıcı deneyimi analiz raporuna göre; karmaşık oyun içi mekaniklerden kaçınılarak, oyuncunun reflekslerine dayalı, klavyedeki boşluk (space) tuşuyla kontrol edilen sade bir oynanış döngüsü kurgulanmıştır. Görsel ve arayüz analizleri

doğrultusunda, karmaşık ve göz yoran tasarımlar yerine Pinterest kaynaklı minimalist 2D sprite tasarımları ile ana menü, oyun ekranı ve oyun sonu panellerinden oluşan kullanıcı dostu bir arayüz şeması benimsenmiştir. Teknik analiz raporu neticesinde ise; projenin nesne yönelimli programlama diline uygun C# mimarisiyle kodlanmasına ve Unity oyun motorunun 2D fizik bileşenleri (Rigidbody2D ve Collider) kullanılarak optimize edilmesine karar verilmiştir.

2.2.2. Düz Metin Açıklamaları

Kullanıcı oyunu başlattığında minimalist tasarıma sahip bir ana menü ekranı ile karşılaşır. Bu ekranda kullanıcıya oyunu başlatma, karakter seçimi yapma veya oyundan çıkış yapma seçenekleri sunulur. Oyuncu "Karakter Seçimi" ekranına

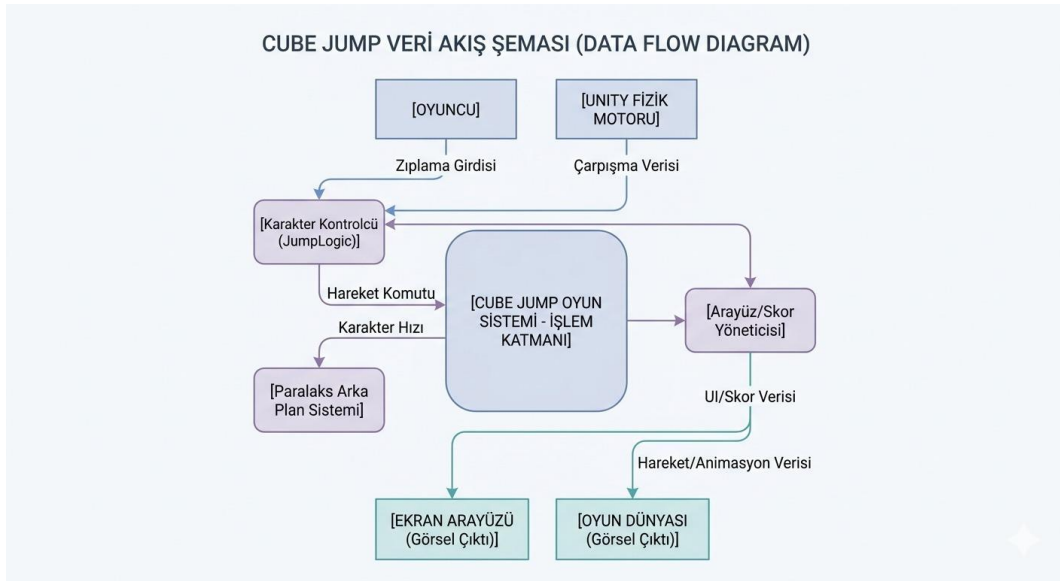
girdiğinde, Pinterest yardımıyla tasarlanmış farklı renk veya görsel özelliklere sahip 2D küp karakterleri arasından dilediğini seçebilir; yapılan bu seçim sistem tarafından hafızaya alınır. Oyuncu ana menüden oyunu başlattığı anda, seçtiği karakter sahnede belirir ve oyun döngüsü aktif hale gelir. Oyuncu, klavyedeki boşluk tuşunu kullanarak seçtiği karakterin zıplama hareketini kontrol eder ve karşısına çıkan engellerin üzerinden atlamaya çalışır. Oyun esnasında, oyuncunun hayatta kaldığı her saniye boyunca kazandığı puan, ekranın üst kısmındaki skor tablosunda anlık ve eş zamanlı olarak güncellenir. Tasarım gereği, zaman ilerledikçe oyunun hızı kademeli olarak artar ve engeller karakterin üzerine daha hızlı gelmeye başlar. Oyuncu herhangi bir engele çarptığı anda Unity fizik motoru çarpışmayı anında algılar ve oyun döngüsünü durdurur. Bu noktada ekrana, o oyun seansında ulaşılan skoru gösteren "Game Over" paneli gelir. Oyuncu bu panel üzerinden isterse tek bir butonla oyunu seçtiği aynı karakterle yeniden başlatabilir, isterse yeni bir karakter seçmek veya çıkış yapmak üzere ana menüye geri dönebilir.

2.2.3. Karar Tablosu

Koşul Kural	R1	R2	R3	R4	R5	R6
Oyuncu Girişi (Ana Menü)	Yanlış	Doğ.	Doğ.	Doğ.	Doğru	Doğru
Karakter Seçimi Yapıldı	-	Yan.	Doğ.	Doğ.	Doğru	Doğru
Oyun Başlatıldı (Oyna)	-	-	Yan.	Doğ.	Doğru	Doğru
Engele Çarpma Var	-	-	-	Yan.	Doğru	Yanlış
Sonuç	Ana Menü Bekleme	Hata1 (Karakter Seçin)	Hata2 (Başlatılamadı)	Oyun Aktif (Skor Artışı)	Game Over Paneli	Rekor/Ödül Ekranı (Veya Ana Menü)

Tablo 2- Oyun Akış ve Karakter Seçim Kontrolü" karar tablosu

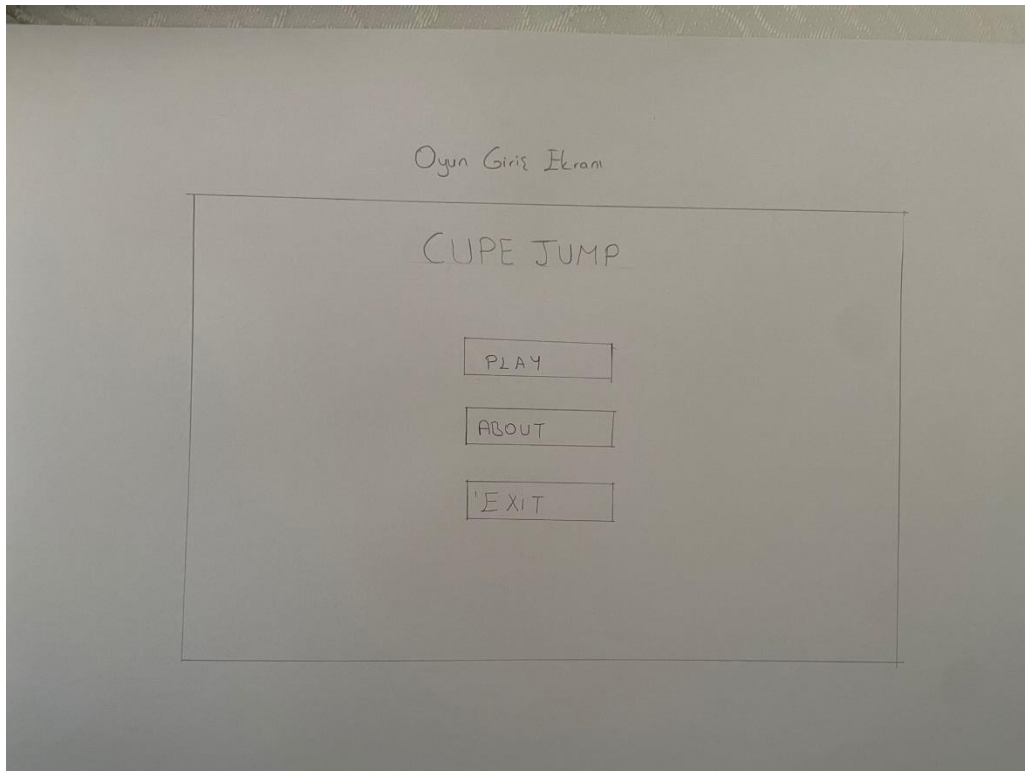
2.2.4. Veri Akış Şeması



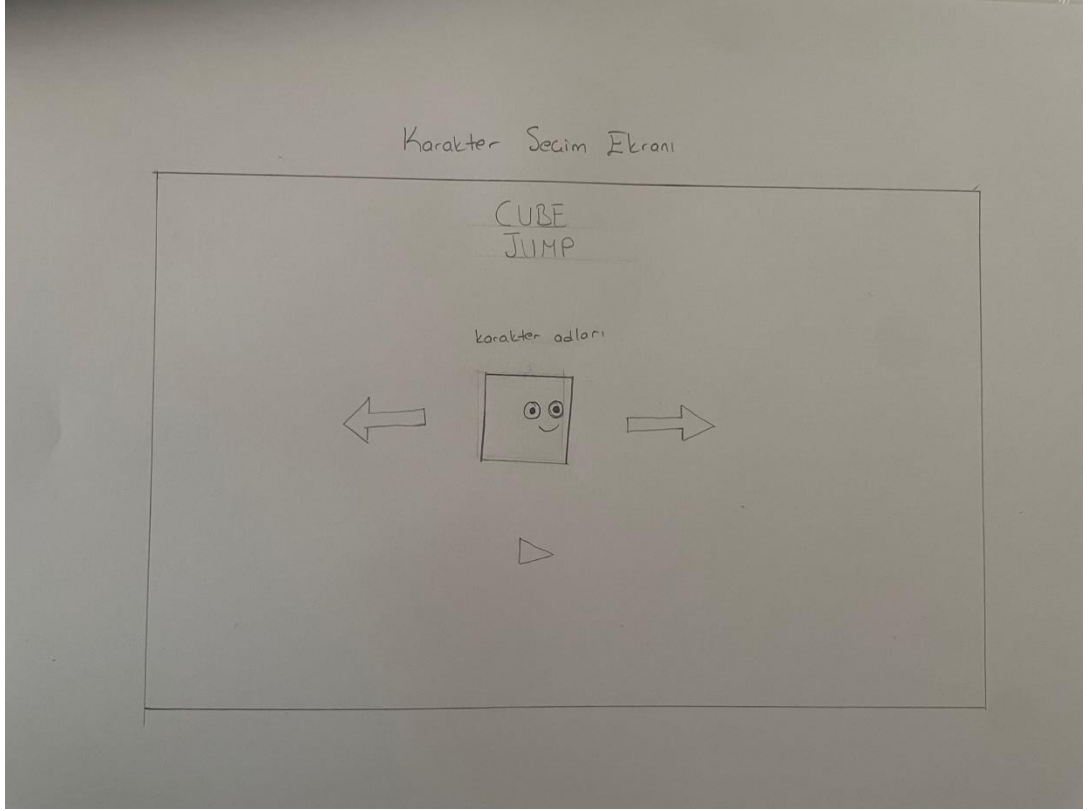
Resim 2-Genel Oyun Şeması

2.3. Tasarım

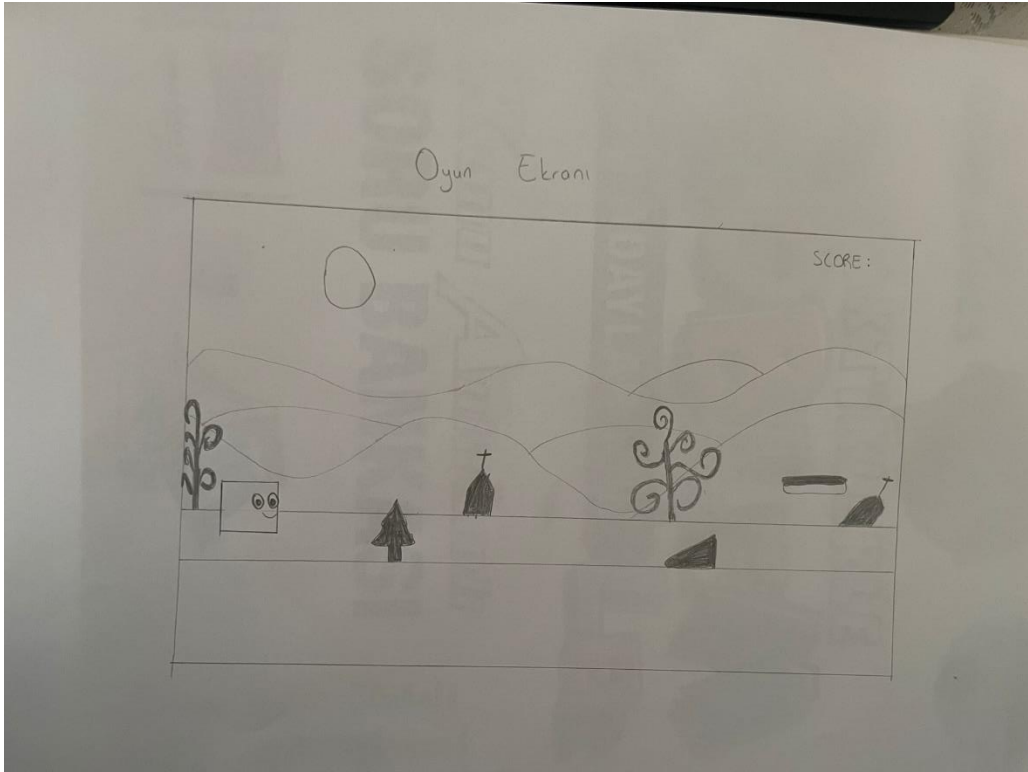
2.3.1. Kağıt Prototip



Resim 3-Oyun Giriş Ekranı



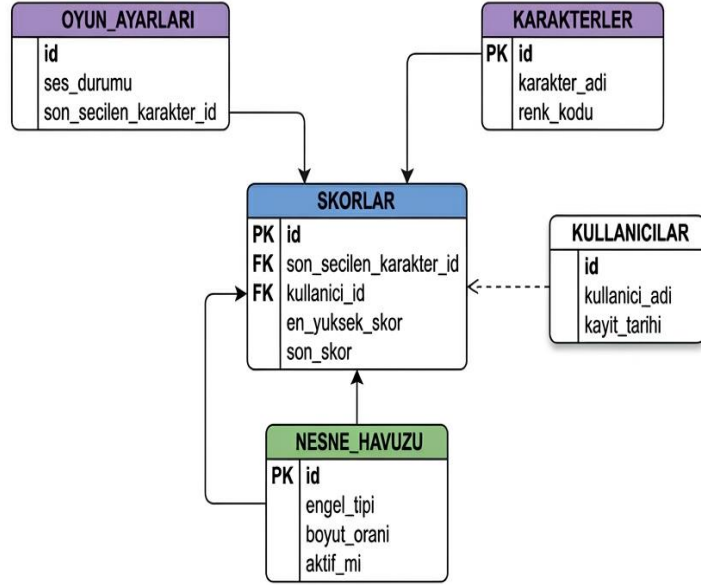
Resim 4-Karakter Seçim Ekranı



Resim 5-Oyun Ekranı

2.3.2.Nesne İlişki Modeli,Veritabanı Tabloları,Veri Sözlüğü

Nesne İlişki Modeli-Varlık İlişki Diyagramı:



Resim 6-Varlık İlişki Diyagramı

Veritabanı Tabloları:

"Cube Jump" projesinin veri mimarisi; dinamik nesne yönetimi, oyuncu tercihleri ve performans kayıtlarının optimize bir biçimde saklanması prensiplerine dayanmaktadır. Sistem mimarisinde veriler; oyun ayarları, karakter tanımlamaları, skor kayıtları ve bellek optimizasyonunu sağlayan nesne havuzu olmak üzere dört ana modül altında kümelенmiştir. İlk olarak OYUN_AYARLARI veri yapısı, kullanıcının ses durumları gibi lokal tercihlerini ve anlık oturumunda tercih ettiği küp modelini son_secilen_karakter_id alanı üzerinden hafızada tutarak kullanıcı deneyiminde süreklilik sağlamaktadır. Oyunda yer alan görsel çeşitliliği yöneten KARAKTERLER modülü ise platformda aktif olarak seçilebilen 4 farklı küp alternatifinin id, karakter_adi ve özelleştirilmiş renk_kodu verilerini barındırmaktadır. Oyun mekaniğinde herhangi bir kilit, satın alma veya ilerlemeli ödül sistemi bulunmadığı için tüm karakter tanımlamaları ilk kurulundan itibaren doğrudan erişilebilir bir mimariyle sunulmuştur. Projenin temel oynanış döngüsünü oluşturan hayatta kalma süresi algoritmalarından elde edilen veriler, SKORLAR modülü altında işlenmektedir. Bu modül sayesinde sistem, oyuncunun cihaz üzerinde kırdığı rekoru en_yuksek_skor alanında muhafaza ederken, son oynanan

seansa ait son_skor verisini de anlık olarak güncelleyip oyuncuya sunabilmektedir. Son olarak projenin teknik omurgasını teşvik eden ve işlemci yükünü minimize eden NESNE_HAVUZU yapısı, sahnede sürekli döngü halinde olan engellerin veri yönetimini üstlenmektedir. Farklı sıralarda ve ölçeklerde spawn edilen engeller, bu modüldeki engel_tipi ve boyut_orani parametreleri ile yapılandırılmakta; nesnelerin aktiflik durumları ise aktif_mi mantıksal alanı üzerinden anlık sorgulanarak Instantiate ve Destroy maliyetlerinin önüne geçmektedir. Sonuç olarak kurgulanan bu veri modeli, oyunun hafif (lightweight) ve cihaz bazlı yapısına uygun, bellek dostu bir nesne ilişkisi sunmaktadır.

Veri Sözlüğü:

Alan Adı	Veri Tipi	Boyut / Format	Boş Olabilir mi? (Null)	Açıklama
id (PK)	Integer	Auto-Increment	Hayır	Tablonun benzersiz anahtarı.
ses_durumu	Boolean	1 bit (0/1)	Hayır	Oyun içi seslerin açık (true) veya kapalı (false) olma durumu.
son_secilen_karakter_id	Integer	-	Hayır	Oyuncunun en son seçtiği küpün ID bilgisini tutan yabancı anahtar.

Tablo 2-Oyun Ayarları Veri Sözlüğü

Alan Adı	Veri Tipi	Boyut / Format	Boş Olabilir mi? (Null)	Açıklama
id (PK)	Integer	Auto-Increment	Hayır	Karakterin benzersiz kimlik numarası.
karakter_adi	VARCHAR	50 karakter	Hayır	Küp karakterinin oyundaki adı (Örn: "Klasik Küp").
renk_kodu	VARCHAR	10 karakter	Hayır	Karakterin Unity içindeki RGB veya Hex renk kodu (Örn: #FF0000).

Tablo 3-Karakterler Veri Sözlüğü

Alan Adı	Veri Tipi	Boyut / Format	Boş Olabilir mi? (Null)	Açıklama
id (PK)	Integer	Auto-Increment	Hayır	Skor kaydının benzersiz kimlik numarası.
kullanici_id (FK)	Integer	-	Evet	Skorun ait olduğu yerel kullanıcı ID'si.
son_skor	Float / Real	-	Hayır	Oyuncunun son oynadığı oyunda hayatta kaldığı süre bazlı skoru.
en_yuksek_skor	Float / Real	-	Hayır	Cihaz hafızasında tutulan, ulaşılan en yüksek rekor süre.

Tablo 4-Skorlar Veri Sözlüğü

Alan Adı	Veri Tipi	Boyut / Format	Boş Olabilir mi? (Null)	Açıklama
id (PK)	Integer	Auto-Increment	Hayır	Havuzdaki engelin benzersiz kimlik numarası.
engel_tipi	VARCHAR	30 karakter	Hayır	Engelin türünü belirten ifade (Örn: "Dikey", "Yatay", "Kısa").
boyut_orani	Float	-	Hayır	Engelin sahnede alacağı random ölçek/boyut çarpanı.
aktif_mi	Boolean	1 bit (0/1)	Hayır	Engelin sahnede görünür (aktif) mi yoksa havuzda yedekte mi olduğu bilgisi.

Tablo 5-Nesne Havuzu Veri Sözlüğü

2.4.Gercekleřtirme/Kodlama

2.4.1. Kullanılan Programlama Dilleri

Cube Jump oyununu geliřtirirken kullanıcı arayüzü tasarımında ve oyun içi mekaniklerde Unity UI (User Interface) ve C# programlama dili teknolojilerini kullandım. Oyun sahnelerinin yapısını, menüleri ve karakter seçme ekranını oluşturmak için Unity'nin hiyerarřı ve arayüz sistemini, bu sayfalar ile oyun içi nesnelerin birbiriyle etkileřime girmesini saęlayan dinamik kodlamalar ve fizik tabanlı mekanikler için ise C# programlama dilini kullandım. Ayrıca nesne havuzlama (Object Pooling) sistemini daha optimize bir hale getirebilmek ve bellek yönetimini doęru kurgulamak için Unity 6 motorunun yerleřik kütüphanelerinden yararlandım. Oyunun veri saklama kısmında ise yerel depolama (PlayerPrefs/Local Veri Yapısı) kullandım. Oyun içerisindeki en yüksek skor, ses ayarları ve seçilen karakter verilerini cihaz hafızası ile baęlarken ise C# dilinin veri yönetim fonksiyonlarını kullandım.

2.4.2. Uygulama Mimarisi

Proje, oyun içi mantıęının ve fizik hesaplamalarının büyük ölçüde C# betikleri (scripts) ile yürütüldüęü, oyuncunun cihaz ekranı üzerinden Unity UI arayüzü ve girdi yöneticileri (Input Manager) aracılıęıyla etkileřime girdięi yerel bir oyun uygulaması (istemci odaklı) mimarisine sahiptir. Harici bir oyun framework'ü veya sunucu katmanı kullanılmamıř; her oyun nesnesi (GameObject) için ona baęlı bileřenler (Components) ve esnek script yapıları tercih edilmiřtir. Bu yapı, optimizasyon ve çalışma performansı açısından kararlı bir bileřen tabanlı (Component-Based) düzendir: tüm sahneler, görsel varlıklar (assets), nesne havuzları ve betikler aynı Unity 6 motoru altında tek bir proje dizininde toplanmıřtır.

Katman	Teknoloji / Yapı
Oyun Motoru (Çalışma Ortamı)	Unity 6 (Editör ortamı, yerleşik bileşen yönetimi ve sahneler arası geçiş sistemleri)
Uygulama ve Mantık Dili	C# (Oyun döngüsü yönetimi, karakter hareketleri, klavye/dokunmatik girdi kontrolleri)
Fizik ve Mekanik Katmanı	Unity 2D Physics (RigidBody2D, BoxCollider2D ile küp ve engeller arası çarpışma/fizik testleri)
Bellek ve Optimizasyon Yönetimi	Object Pooling - Nesne Havuzlaması (Sürekli engel üretimi maliyetini önlemek amacıyla pasif/aktif nesne yönetimi)
Yerel Depolama (Veri Katmanı)	PlayerPrefs / Local JSON (En yüksek skor rekoru, son seçilen karakter indis değeri ve ses ayarlarının cihaz hafızasında tutulması)
Sunum ve Arayüz Katmanı	Unity UI - Canvas (Giriş menüsü, buton tasarımları, anlık akan hayatta kalma süresi skoru ve karakter seçme paneli)

Tablo 6-Kullanılan Teknolojiler

2.4.3.Kodlama Süreci

Projenin kodlama sürecinde önceliğimi temel oyun mekaniklerine ve fizik tabanlı sistemlerin kurgulanmasına (core gameplay) verdim. Bu proje üzerinde tek başıma çalıştığım için daha düzenli bir şekilde ilerlemem gerekiyordu; bu durum, geliştirme aşamasında karşılaştığım olası bug ve optimizasyon hatalarını çok daha rahat çözmemde etkili oldu. İlk olarak C# dilinde küpün hareket ve zıplama algoritması gibi temel fizik kodlarını yazarak işe başladım, ardından oyunun omurgasını oluşturacak olan nesne havuzlama (Object Pooling) sistemini kurguladım. Bu havuzu test etmek ve yazarken bir yandan dinamikleri anlık olarak görebilmek adına oyun sahnesine farklı boyutlarda ve rastgele sıralarda spawn olan örnek engeller dahil ettim. Bu temel mekanikleri kararlı bir şekilde çalıştırıp oyun döngüsünü oturtuktan sonra kullanıcı arayüzü (UI) ve görsel yönetim kısmına geçtim. Unity'nin yerleşik Canvas sistemini, C# script'lerini ve PlayerPrefs yerel depolama birimini bir arada kullanarak yazmaya başladım. C# dilini, arayüzdeki buton etkileşimlerini yönetmek ve karakter seçme ekranı ile oyun sahneleri arasında veri bağına kurmak için kullandım. Karakter seçimi aşamasında, oyuna mekanik bir etkisi olmayan 4 farklı küp modelini sisteme dahil ederek testlerimi gerçekleştirdim. Aynı zamanda arayüzün daha esnek, minimalist ve kullanıcı dostu bir tasarıma sahip olması için Unity'nin UI kütüphanelerinden yararlandım; oyun içindeki anlık akan süre bazlı skor göstergesi ve ses kontrol paneli ikonlarını da bu doğrultuda yapılandırdım.

2.5.Bakım/Destek

2.5.1.Olası Güncelleme Süreçleri

"Cube Jump" projesinin sürdürülebilirliğini artırmak, oyuncu bağlılığını güçlendirmek ve projenin teknik kapsamını genişletmek amacıyla gelecek dönemde sisteme dahil edilmesi planlanan olası güncelleme seçenekleri belirlenmiştir. Bu doğrultuda ilk olarak, mevcut "tek hatada yanma" mekanizmasını esneterek oyun derinliğini artırmak adına bir can sistemi entegre edilmesi planlanmaktadır; böylece oyuncular topladıkları nesnelerle veya belirli eşiklerde ek haklar kazanarak oyuna devam edebilecektir. İkinci olarak, projenin hedef kitlesini genişletmek amacıyla oyun mimarisi mobil platformlara (Android ve iOS) uyumlu hale getirilecek, dokunmatik ekran girdileri ve ivme ölçer kontrolleri sisteme dahil edilecektir. Görsel çeşitliliği ve kişiselleştirmeyi artırmak adına, oyuna mekaniksel olarak da farklılıklar sunan (hız, zıplama yüksekliği veya küçülme özellikleri gibi) çeşitli yeni karakter modelleri ve bu karakterlerin oyun içi puanlarla edinilebileceği bir mağaza yapısı kurulacaktır. Son olarak oyun skorları üzerinde kapsamlı bir geliştirme yapılarak, yerel hafızadaki skor takibi yerine çevrimiçi bir veri tabanı üzerinden çalışan, oyuncuların dünya genelindeki diğer kullanıcılarla rekabet etmesine olanak tanıyan küresel bir liderlik tablosu ve başarımlar sistemi entegre edilecektir. Bu güncellemeler sayesinde projenin hem optimizasyon kapasitesi test edilecek hem de ticari bir mobil oyun standartlarına ulaştırılması sağlanacaktır.

2.5.2. Hata Düzeltme Mekanizmaları

"Cube Jump" projesinin akıcı çalışması ve FPS düşüşlerinin önlenmesi amacıyla test süreçlerine geniş yer ayrılmıştır. Testler sırasında, sürekli üretilen engellerin Unity hiyerarşi panelinde yığılarak belleği zorladığı tespit edilmiştir. Bu performans sorununu çözmek için kameranın görüş alanı dışına görünmez bir "engel yakalama alanı" (Trigger) yerleştirilmiştir; böylece ekrandan çıkan nesneler bu sınıra çarptığı anda deaktif edilerek bellek yükü ortadan kaldırılmıştır. Farklı süre ve hızlarda yapılan bu testlerin yanı sıra proje 3 ayrı cihazda daha denenmiştir. Bu sayede oyun arayüzünün (UI) farklı ekran çözünürlüklerinde ve en-boy oranlarında kayma yapmadan, kararlı ve esnek çalışması güvence altına alınmıştır. Karşılaşılan diğer yapısal hatalar ve çözümleri sonuç bölümünde detaylandırılacaktır.

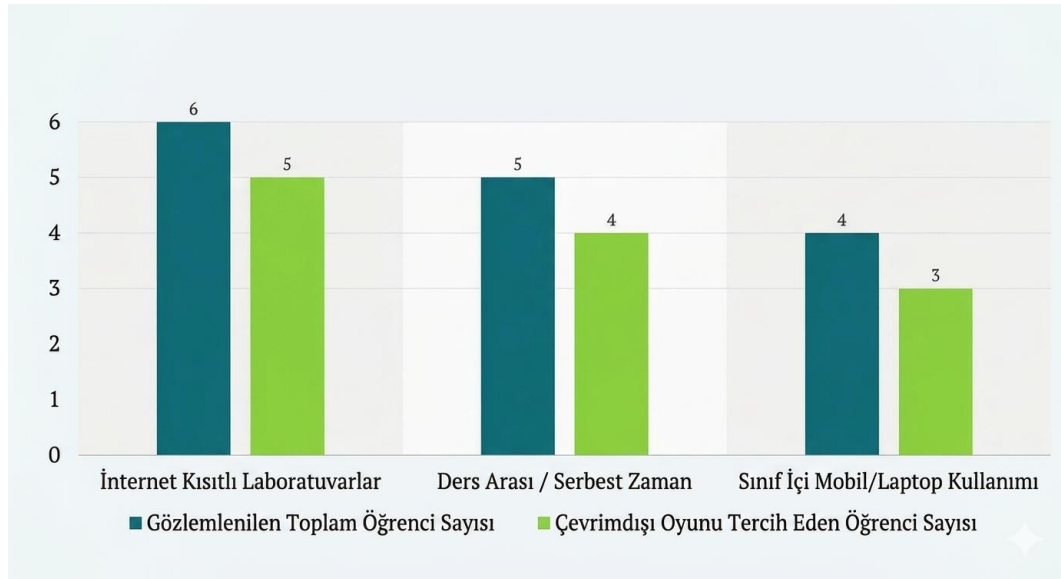
2.5.3. Sistemin Sürdürülebilirliği

"Cube Jump" projesi, hafif mimarisi ve bileşen tabanlı yapısı sayesinde minimum sistem kaynağı tüketen, bakımı kolay ve genişletilmeye açık bir oyun uygulamasıdır. Projenin yerel (local) mekaniklere dayanması ve modüler kod dizilimi, ilerleyen süreçlerde yeni karakter modellerinin, harici platform entegrasyonlarının veya ek mekaniklerin sisteme büyük maliyetler gerektirmeden kolayca eklenmesini mümkün kılmaktadır. Büyük bütçeli sunucu altyapılarına ihtiyaç duymayan bu yapı, hem geliştirici tarafında düşük maliyetli bir yönetim süreci sunmakta hem de son kullanıcının cihazını yormadan performanslı bir şekilde çalışabilmektedir. Oynanış tarafında ise hızlı ve sade arayüz tasarımıyla kullanıcılara akıcı bir deneyim yaşatmayı, refleks tabanlı döngüsüyle de uzun vadeli bir oynanabilirlik sunmayı hedeflemektedir.

3. SONUÇ

3.1. Elde Edilen Bulgular

"Cube Jump" projesi, internet erişimi kısıtlı eğitim ortamlarında öğrencilerin çevrimdışı vakit geçirmek için alternatif bulamaması ve klasik tarayıcı oyunlarına yönelmesi gözlemlenerek kurgulanmıştır. Sektördeki büyük bütçeli yapımların sürekli internet, yüksek donanım ve reklam barındırma zorunlulukları; bağımsız projelerin ise optimizasyon ve karmaşık arayüz sorunları kullanıcı deneyimini olumsuz etkilemektedir. Bu eksiklikleri gidermek amacıyla geliştirilen oyun; tamamen çevrimdışı çalışabilen, minimalist ve performans odaklı bir yapıya sahiptir. Böylece, düşük donanımlı cihazlarda bile internet gereksinimi olmadan, reklamsız ve akıcı bir oyun deneyimi sunulması hedeflenmiştir.



Şekil 2-Çevrimdışı Ortamlarda Oyun Tercihi Oranı

3.2. Proje Hedefine Ulařımı

"Cube Jump" projesi, bařlangıçta hedeflenen tüm teknik ve işlevsel amaçlarına başarıyla ulaşmıştır. İnternet erişimi kısıtlı ortamlarda kullanıcıların hafif, akıcı ve reklamsız bir çevrimdışı oyun ihtiyacı tam anlamıyla karşılanmıştır. Geliştirme sürecinde ortaya çıkan bellek yığılması sorunu, ekran dışına kurulan "engel yakalayıcı (Trigger)" mekanizmasıyla çözülerek cihazı yormayan, yüksek performanslı bir optimizasyon elde edilmiştir. Farklı cihazlarda yapılan testlerle de arayüzün kararlılığı kanıtlanmış; proje hem modüler kod yapısı hem de kullanıcı odaklı minimalist tasarımıyla eksiksiz bir şekilde tamamlanmıştır.

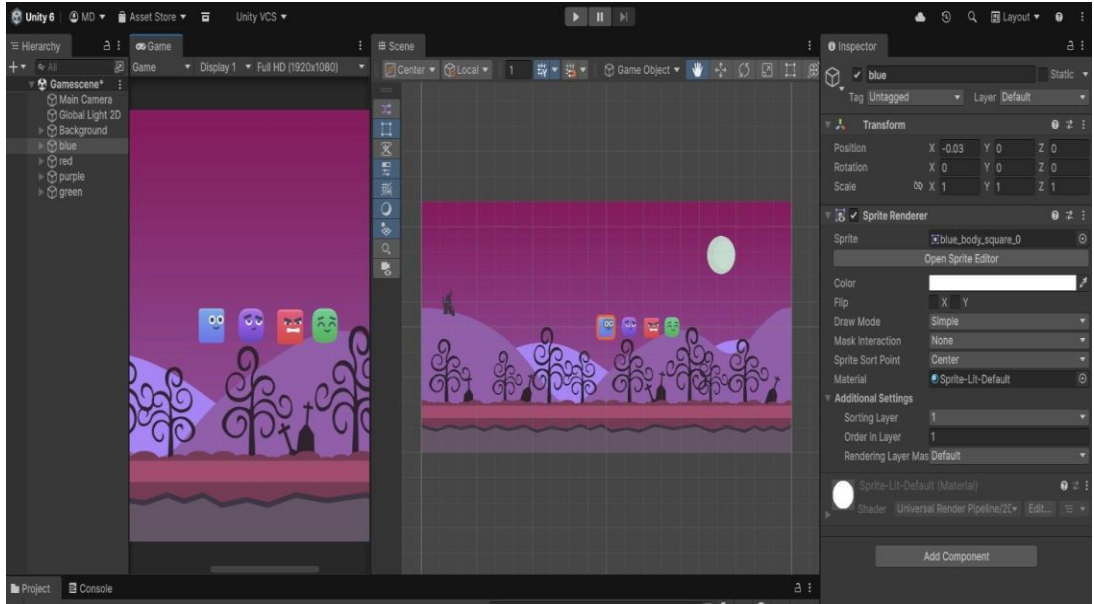
3.3.Karşılasılan Sorunlar ve Çözüm Önerileri

"Cube Jump" projesinin geliştirme ve test aşamalarında iki temel teknik sorunla karşılaşılmış ve bunlara yönelik optimizasyon çözümleri üretilmiştir. İlk olarak, oyun döngüsü içinde sürekli üretilen engellerin ekran dışına çıktıktan sonra bellekten silinmediğı, Unity hiyerarři panelinde yığılarak işlemciyi ve RAM'i aşırı zorladığı tespit edilmiştir. Bu performans kaybını ve FPS düşüşlerini engellemek adına, ana kameranın görüş açısının hemen arkasına görünmez bir "engel yakalama katmanı" (Trigger Area) yerleştirilmiştir; böylece işlevini yitiren nesneler bu sınıra çarptığı anda deaktif edilerek bellek yükü tamamen ortadan kaldırılmıştır. İkinci olarak, projenin farklı donanımlara sahip cihazlardaki testlerinde, ekran çözünürlükleri ve en-boy oranları (aspect ratio) değıştikçe skor göstergesi, butonlar ve ikonlar gibi arayüz elemanlarında kaymalar yaşandığı gözlemlenmiştir. Bu görsel hatayı çözmek amacıyla Unity Canvas Scaler bileşeni sabit piksel modundan "ekran boyutuna göre ölçekleme" moduna alınmış ve tüm UI elemanları ekran köşelerine doğru şekilde çapalanarak (anchor) arayüzün her cihazda kararlı çalışması sağlanmıştır.

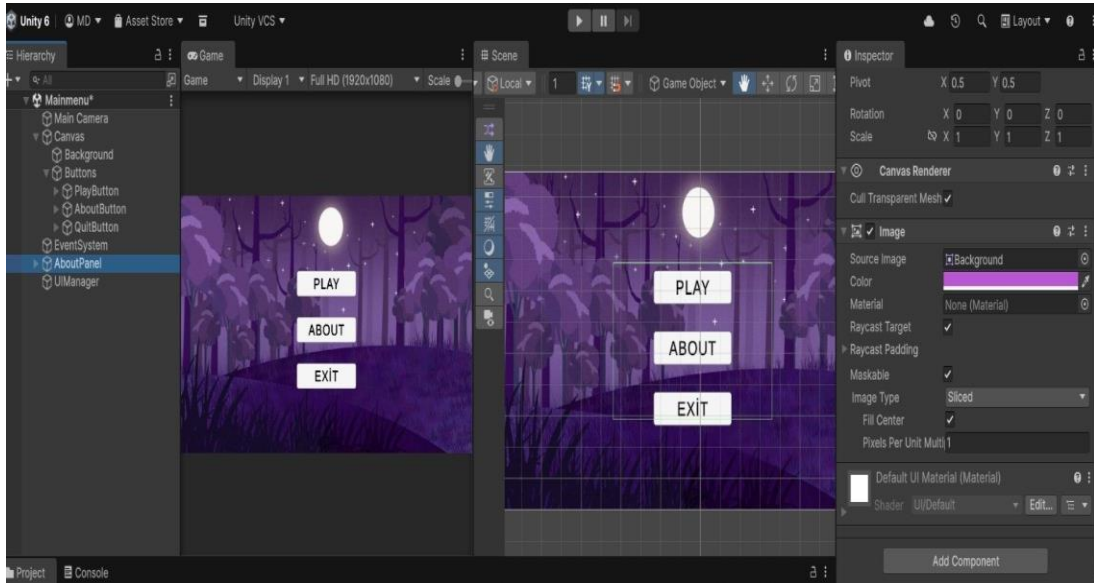
4. KAYNAKÇA

- 1- Yazılım Yaşam Döngüsü ve Scrum, Medium, <https://medium.com/@yunus.selcuk/yaz%C4%B1%C4%B1m-ya%C5%9Fam-d%C3%B6ng%C3%BCs%C3%BC-ve-scrum-2770a183dd7d> ,23 Eylül 2019
- 2- Unity Technologies, (2024), Unity 6 (Versiyon 6000.0) [Oyun Motoru], <https://unity.com/>
- 3- Microsoft Corporation, (2024), C# [Programlama Dili], <https://learn.microsoft.com/en-us/dotnet/csharp/>
- 4- Pinterest, (2024), Pinterest [Görsel Tasarım ve İlham], <https://tr.pinterest.com/>
- 5- Google, (2024), Gemini [Yapay Zeka Modeli], <https://gemini.google.com/>
- 6- Unity Technologies, (2024), Unity Asset Store [Oyun Varlıkları ve Paketleri], <https://assetstore.unity.com/>
- 7- Pixabay, (2024), Pixabay Audio [Oyun İçi Müzik ve Ses Efektleri], <https://pixabay.com/music/search/game%20loop/>

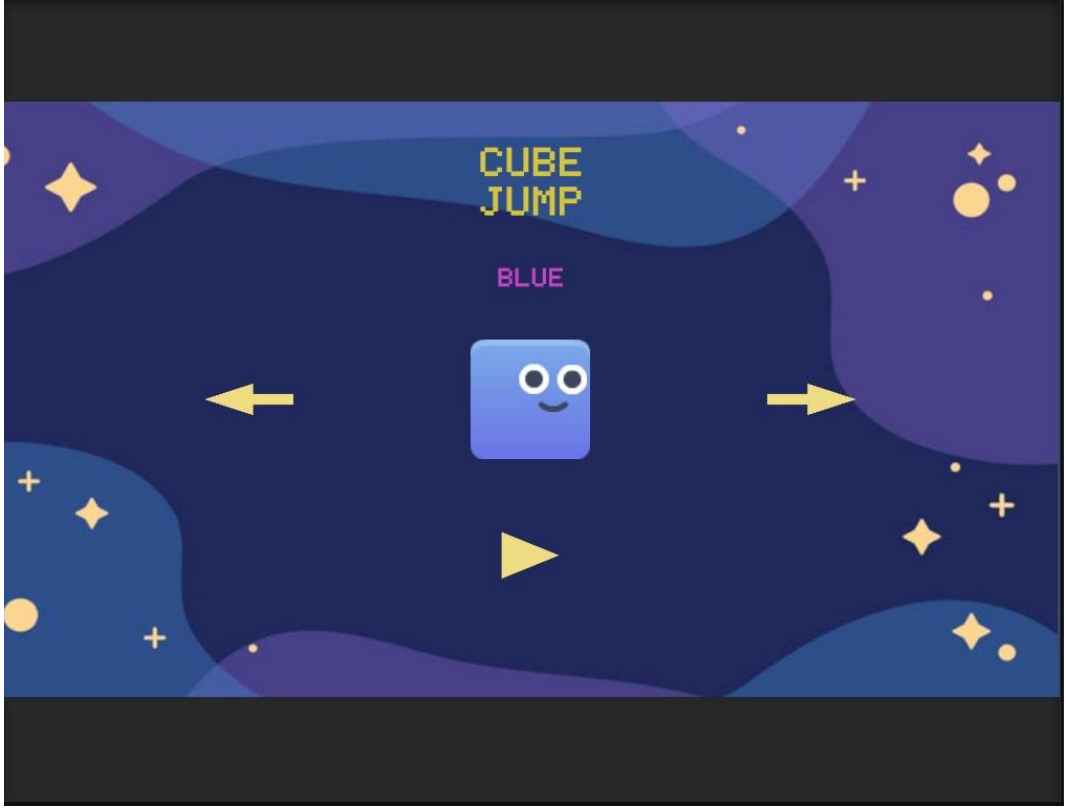
5. UYGULAMAYA AİT EKRAN GÖRÜNTÜLERİ



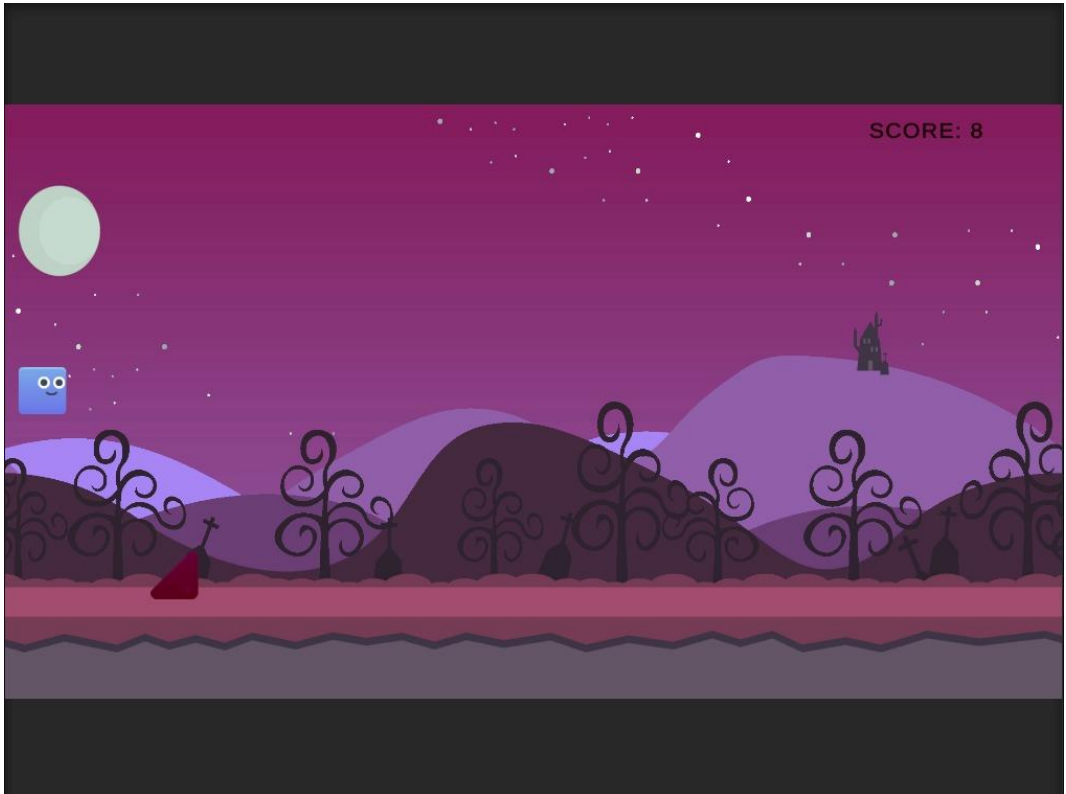
Resim 7-Karakter Tasarım



Resim 8-Giriş Ekranı Tasarımı



Resim 9-Karakter Seçim Ekranı



Resim 10-Oyun Ekranı

6. UYGULAMAYA AİT KODLAR

```
using UnityEngine;
using UnityEngine.SceneManagement;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class MainMenu : MonoBehaviour
{
    public void StartGame()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex +
1);
    }

    public void ExitGame()
    {
        //UnityEditor.EditorApplication.isPlaying = false;
        Application.Quit();
    }

    public void BackToMenu()
    {
        SceneManager.LoadScene(0);
    }

}
```

Bu kod, Unity oyun motorunda oyunun Ana Menü ve Sahneler Arası Geçiş Sistemini kontrol eden temel C# betiğidir (script). Oyundaki butonlara (Başla, Çıkış, Menüye Dön) tıklandığında ne yapılacağını belirler.

```

using System.Collections;
using UnityEngine;
using TMPro;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class SpawnObstacle : MonoBehaviour
{
    [SerializeField] GameObject[] obsatclePrefab;
    [SerializeField] TMP_Text scoreText;
    float initialSpeed = 10f;
    float maxSpeed = 35f;
    float speedIncreaseRate = 1f;
    float timeElapsed = 0f;
    int score = 0;
    float scoreIncreaseRate = 1f;
    void Start()
    {
        StartCoroutine(SpawnObstacles());
        StartCoroutine(DelayedOperation());

    }
    IEnumerator SpawnObstacles()
    {
        while(true)
        {
            float randomDelay = Random.Range(0.7f, 3f);
            yield return new WaitForSeconds(randomDelay);
            int i = Random.Range(0, obsatclePrefab.Length);
            Vector3 spawnPosition = transform.position;
            if(i==1 )
            {
                spawnPosition.y += 1;
            }
            GameObject obstacle =
Instantiate(obsatclePrefab[i],spawnPosition,Quaternion.identity);
            Obstacle obstacleScript=obstacle.GetComponent<Obstacle>();
            if(obstacleScript !=null)
            {
                obstacleScript.SetSpeed(initialSpeed);
            }
        }
    }
}

```

```

    }

}

}

void IncreaseSpeed()
{
    initialSpeed += speedIncreaseRate;
    if(initialSpeed>maxSpeed)
    {
        initialSpeed = maxSpeed;
    }
    scoreIncreaseRate = initialSpeed / 10f;
}
IEnumerator DelayedOperation()
{
    while(true)
    {
        score += 1;
        scoreText.text = "SCORE: " + score.ToString();
        yield return new WaitForSeconds(1f / scoreIncreaseRate);
    }
}

private void FixedUpdate()
{
    timeElapsed += Time.deltaTime;
    if(timeElapsed>=5f)
    {
        timeElapsed = 0;
        IncreaseSpeed();
    }
}
}

```

Bu kod, oyununun kalbi olan "Sonsuz Engel Üretme (Spawning), Skor ve Zorluk Derecesi (Hız) Artırma" mekanizmasını yöneten ana script'tir. Oyun süresi

uzadıkça engellerin daha hızlı gelmesini ve skorun daha hızlı artmasını sağlayarak oyunu gitgide zorlaştırır.

```
using UnityEngine;

*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class EnemyDestroyer : MonoBehaviour
{
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy"))
        {
            Destroy(collision.gameObject);
        }
    }
}
```

Bu kod, oyunun arka planında çalışan ve performansı koruyan (RAM/işlemci kasmaını engelleyen) görünmez bir temizlik mekanizmasıdır. Kameranın görüş alanından çıkıp sol tarafa doğru giden ve artık oyuncu için hiçbir tehdit oluşturmayan engelleri (kodda "Enemy" tag'i verilmiş nesneleri) otomatik olarak yok eder.

```
using UnityEngine;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class Obstacle : MonoBehaviour
{
    float speed = 10f;

    public void SetSpeed(float newSpeed)
    {
        speed = newSpeed;
    }
    // Update is called once per frame
    void Update()
    {
        Debug.Log(speed);
        transform.Translate(Vector3.left * Time.deltaTime * speed);
    }
}
```

Bu kod, oyunda üretilen her bir engelin (Obstacle) ekranda sağdan sola doğru hareket etmesini ve oyunun yöneticisinden (SpawnObstacle script'inden) hız bilgisini almasını sağlayan hareket mekanizmasıdır.

```

using NUnit.Framework;
using UnityEngine;
using System.Collections.Generic;
using TMPro;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class CharacterSelection : MonoBehaviour
{
    public List<GameObject> characters = new List<GameObject>();
    public TextMeshProUGUI characterName;
    public Button selectButton;
    public static int currentIndex = 0;
    void Start()
    {
        UpdateCharacterDisplay();
        selectButton.onClick.AddListener(OnSelectButtonClicked);
    }
    void UpdateCharacterDisplay()
    {
        foreach(var character in characters)
        {
            character.SetActive(false);
        }
        characters[currentIndex].SetActive(true);
        characterName.text = characters[currentIndex].name;
    }
    public void OnRightButtonClicked()
    {
        currentIndex = (currentIndex + 1) % characters.Count;
        UpdateCharacterDisplay();
    }
    public void OnLeftButtonClicked()
    {
        currentIndex = (currentIndex - 1) % characters.Count;
        UpdateCharacterDisplay();
    }
    void OnSelectButtonClicked()

```

```

    {
        SceneManager.LoadScene(2);
        PlayerPrefs.SetInt("SelectedCharacterIndex", currentIndex);
        PlayerPrefs.Save();
    }
}

```

Oyuncunun sağ ve sol butonlara basarak karakterler arasında geçiş yapmasını, seçtiği karakterin adını ekranda görmesini ve "Seç" butonuna bastığında bu seçimi hafızaya kaydederek sonraki sahneye (örneğin oyunun asıl başladığı sahneye) geçmesini sağlar.

```

using UnityEngine;
using UnityEngine.SceneManagement;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class CharacterSpawner : MonoBehaviour
{
    public GameObject[] characters;
    int index = 0;
    void Start()
    {
        index = CharacterSelection.currentIndex;
        Debug.Log("Index:" + index);
        int selected = index;
        if(selected >= 0 && selected < characters.Length)
        {
            GameObject selectedCharacterPrefab = characters[selected];
            Instantiate(selectedCharacterPrefab, transform.position, Quaternion.identity);
        }
        else
        {
            Debug.Log("Index değeri hatalı");
        }
    }
    public void RestartGame()
    {
        SceneManager.LoadScene(1);
        Time.timeScale = 1.0f;
    }
}

```

Görevi, oyuncu asıl oyun sahnesine geçtiğinde, seçtiği karakteri oyun alanında yaratmaktır (Spawn etmek).

```
using UnityEngine;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class NewMonoBehaviourScript : MonoBehaviour
{
    float jumpPower = 16f;
    [SerializeField] private Rigidbody2D rb;
    [SerializeField] private Transform groundCheck;
    [SerializeField] private LayerMask groundLayer;

    [SerializeField] GameObject panel;
    [SerializeField] GameObject restartButton;
    [SerializeField] AudioSource bgAudioSource;
    [SerializeField] AudioSource dieSource;
    GameObject backGround;
    void Start()
    {
        panel = GameObject.Find("Panel");
        restartButton = GameObject.Find("RestartButton");
        backGround = GameObject.Find("BackGround");
        bgAudioSource=backGround.GetComponent<AudioSource>();

        panel.SetActive(false);
        restartButton.SetActive(false);
    }
    void Update()
    {
        if(Input.GetButtonDown("Jump")&& IsGrounded())
        {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpPower);
        }
        if(Input.GetButtonUp("Jump")&& rb.linearVelocity.y>0f)
        {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y
* 0.5f);
        }
    }
    bool IsGrounded()
```



```

    {
        return Physics2D.OverlapCircle(groundCheck.position, 0.2f,
groundLayer);
    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy"))
        {
            dieSource.Play();
            bgAudioSource.Stop();
            panel.SetActive(true);
            restartButton.SetActive(true);
            Time.timeScale = 0f;
        }
    }
}

```

Karakter Hareketini (Zıplama Mekaniğini) ve Karakterin Ölme/Yenilme Senaryosunu yöneten ana koddur.

```

using UnityEngine;
*****

Ad Soyad: Melike Duman
Öğrenci Numarası:262484053
*****

public class Parrallax : MonoBehaviour
{
    Material _mat;
    float distance;
    float timeSinceLastIncrease = 0;
    [SerializeField] float initialSpeed;
    [SerializeField] float speedIncreaseAmount = 0.01f;
    [SerializeField] float maxSpeed;

    MonoBehaviour is created
    void Start()
    {
        _mat= GetComponent<Renderer>().material;
    }
}

```

```
void Update()
{
    timeSinceLastIncrease += Time.deltaTime;
    if(timeSinceLastIncrease>=10f)
    {
        initialSpeed += speedIncreaseAmount;
        initialSpeed = Mathf.Min(initialSpeed, maxSpeed);
        timeSinceLastIncrease = 0f;
    }
    distance += Time.deltaTime * initialSpeed;
    _mat.SetTextureOffset("_MainTex", new Vector2(distance, 0));
}
}
```

Bu script, oyundaki derinlik ve sürekli hareket algısını oluşturmak amacıyla geliştirilmiş bir parallax efekt mekanizmasıdır. Arka plan nesnesini haritada fiziksel olarak kaydırmak yerine, nesne üzerindeki materyalin doku ofset değerini (SetTextureOffset) X ekseninde dinamik olarak değiştirir. Ayrıca ana oyun döngüsündeki zorluk artışına paralel olarak, arka planın akış hızı da her 10 saniyede bir kademeli olarak artırılır ve belirlenen maksimum hız değerinde (maxSpeed) dengelenerek kararlı bir görsel akış sunulur.